

1. DESCRIPCIÓN GENERAL DEL SISTEMA

El presente proyecto corresponde al desarrollo de un sistema distribuido para la gestión de turnos utilizando una arquitectura basada en microservicios. La solución fue construida aplicando los conceptos abordados durante el curso de Sistemas Distribuidos, integrando mecanismos de comunicación distribuida, persistencia desacoplada, contenerización, monitoreo y tolerancia a fallos.

El propósito principal del sistema consiste en administrar el proceso de registro de usuarios y asignación automática de turnos mediante componentes independientes que interactúan entre sí utilizando protocolos HTTP/REST. Adicionalmente, el sistema incorpora mecanismos de notificación y consolidación de información mediante un servicio especializado encargado de integrar datos provenientes de múltiples microservicios.

La problemática principal que motivó el desarrollo de esta solución se relaciona con las limitaciones presentes en arquitecturas monolíticas tradicionales. En este tipo de arquitecturas, todas las funcionalidades se encuentran centralizadas dentro de una única aplicación, generando altos niveles de acoplamiento, dificultades de mantenimiento y problemas de escalabilidad. Además, un fallo interno puede comprometer completamente la disponibilidad del sistema.

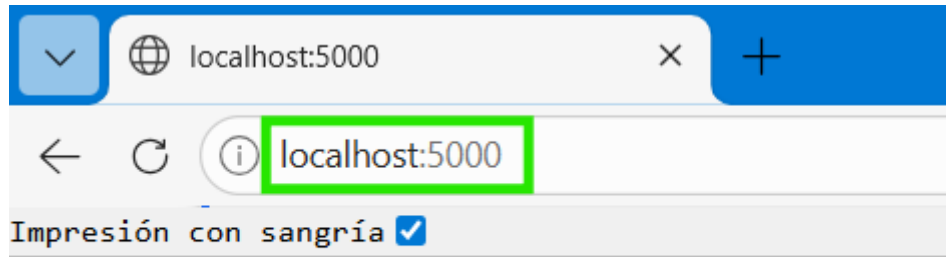
Con el objetivo de reducir estas limitaciones, se implementó una arquitectura distribuida basada en microservicios, donde cada servicio administra una responsabilidad específica y se ejecuta de forma independiente dentro de contenedores Docker. Esto permite mejorar la modularidad, facilitar el mantenimiento y reducir el impacto de posibles fallos internos.

El sistema desarrollado permite registrar usuarios, generar turnos automáticamente, almacenar notificaciones y consolidar información proveniente de diferentes componentes distribuidos. Además, incorpora mecanismos de monitoreo básico y tolerancia a fallos mediante la implementación del patrón Circuit Breaker, el cual permite controlar el consumo de servicios que presenten indisponibilidad temporal.

Desde el punto de vista funcional, el sistema se encuentra dirigido a escenarios de atención donde se requiere organizar procesos de asignación de turnos para usuarios, permitiendo validar información, generar registros persistentes y consultar el estado general de la operación distribuida.

El alcance actual del proyecto corresponde a una implementación académica funcional orientada a demostrar el comportamiento de una arquitectura distribuida real. Aunque el sistema implementa funcionalidades básicas, incorpora componentes fundamentales utilizados en entornos empresariales modernos, como API Gateway, comunicación desacoplada, monitoreo distribuido y mecanismos de recuperación ante fallos.

- Gateway funcionando:



```
{
  "mensaje": "API Gateway funcionando",
  "servicios": [
    "users-service",
    "turns-service",
    "notifications-service"
  ]
}
```

- Contenedores ejecutándose:

```
C:\Users\ASUS>docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
a6b1fe781a9d	gestor-turnos-gateway-service		"python app.py"	48 minutes ago	Up 48 minutes	0.0.0.0:5
000->5000/tcp,	[::]:5000->5000/tcp	gestor-turnos-gateway-service-1				
6d45fb336914	gestor-turnos-reports-service		"python app.py"	48 minutes ago	Up 48 minutes	0.0.0.0:5
004->5000/tcp,	[::]:5004->5000/tcp	gestor-turnos-reports-service-1				
e204c14542fd	gestor-turnos-turns-service		"python app.py"	48 minutes ago	Up 48 minutes	0.0.0.0:5
002->5000/tcp,	[::]:5002->5000/tcp	gestor-turnos-turns-service-1				
ef4a85848fcd	gestor-turnos-users-service		"python app.py"	48 minutes ago	Up 48 minutes	0.0.0.0:5
001->5000/tcp,	[::]:5001->5000/tcp	gestor-turnos-users-service-1				
eb78e11f2958	gestor-turnos-notifications-service		"python app.py"	48 minutes ago	Up 48 minutes	0.0.0.0:5
003->5000/tcp,	[::]:5003->5000/tcp	gestor-turnos-notifications-service-1				
f1e4b9af2a13	postgres:15		"docker-entrypoint.s..."	48 minutes ago	Up 48 minutes	0.0.0.0:5
432->5432/tcp,	[::]:5432->5432/tcp	postgres_db				

```
C:\Users\ASUS>
```

2. ARQUITECTURA FINAL DEL SISTEMA

El sistema fue desarrollado utilizando una arquitectura basada en microservicios, donde cada componente cumple una responsabilidad específica y se ejecuta de manera independiente dentro de contenedores Docker. Esta arquitectura permite desacoplar las funcionalidades principales del sistema y facilitar la comunicación distribuida entre servicios.

La solución implementada está compuesta por los siguientes componentes:

- Gateway Service
- Users Service
- Turns Service
- Notifications Service
- Reports Service
- PostgreSQL

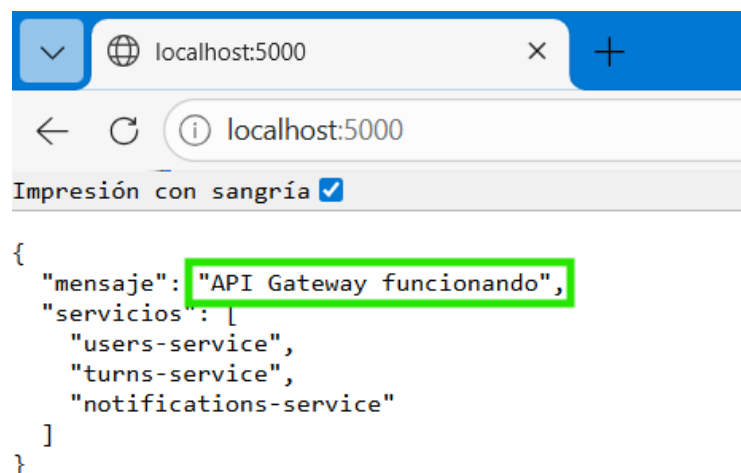
2.1. Gateway Service

El Gateway Service funciona como punto único de entrada al sistema. Todas las solicitudes realizadas desde Postman o navegador son recibidas inicialmente por este componente y posteriormente redireccionadas hacia el microservicio correspondiente.

Puerto utilizado: 5000

Endpoints principales:

- GET /users
- POST /users
- GET /turns
- POST /turn
- GET /notifications
- POST /notify



2.2. Users Service

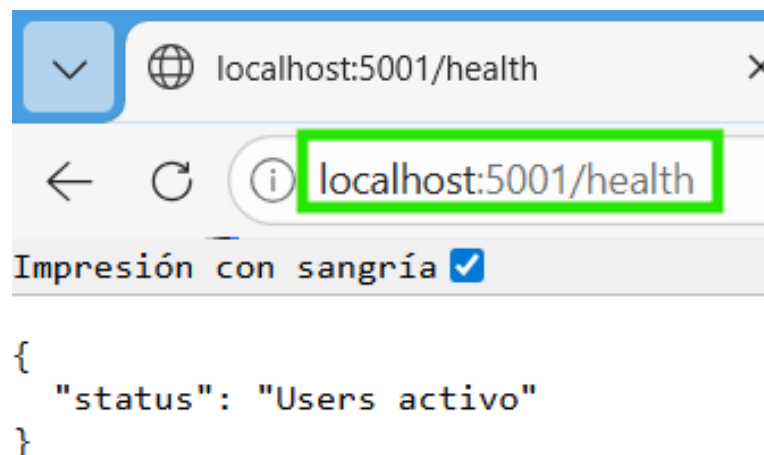
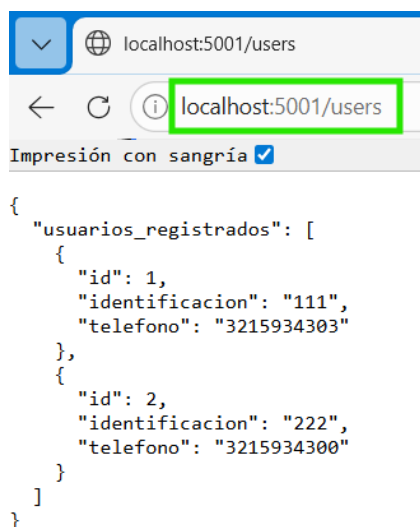
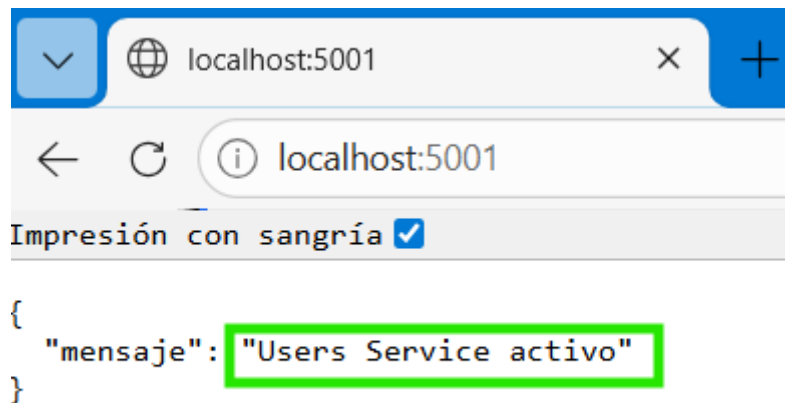
Este microservicio administra el registro y consulta de usuarios. Su función principal consiste en validar y almacenar la información de los usuarios dentro del sistema.

Puerto utilizado: 5001

Base de datos asociada: users_db

Endpoints implementados:

- GET /users
- POST /users
- GET /health



2.3. Turns Service

El Turns Service administra la generación automática de turnos y representa uno de los principales puntos de integración de la arquitectura.

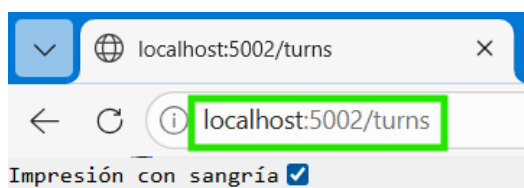
Antes de crear un turno, este servicio consulta al Users Service para validar que el usuario exista. Posteriormente, realiza una solicitud HTTP hacia Notifications Service para registrar la notificación correspondiente.

Puerto utilizado: 5002

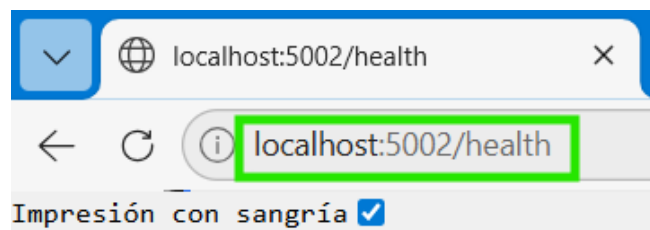
Base de datos asociada: turns_db

Endpoints implementados:

- GET /turns
- POST /turn
- GET /health



```
{
  "turnos": [
    {
      "id": 1,
      "identificacion": "111",
      "turno": "T1"
    },
    {
      "id": 2,
      "identificacion": "222",
      "turno": "T2"
    }
  ]
}
```



```
{
  "status": "Turns activo"
}
```

2.4. Notifications Service

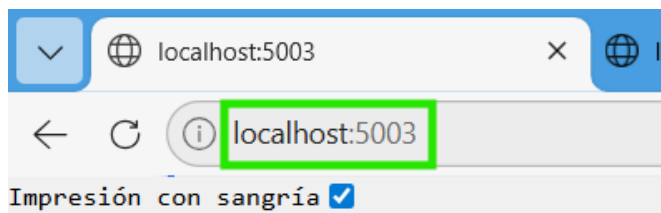
Este servicio administra el almacenamiento de notificaciones generadas por el sistema después de crear un turno.

Puerto utilizado: 5003

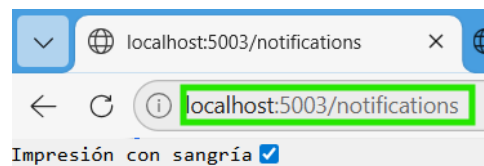
Base de datos asociada: notifications_db

Endpoints implementados:

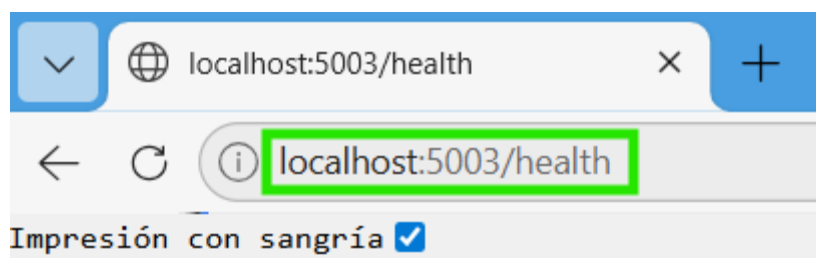
- GET /notifications
- POST /notify
- GET /health



```
{
  "mensaje": "Notifications Service activo"
}
```



```
{
  "mensaje": "Listado de notificaciones",
  "notificaciones": [
    {
      "id": 1,
      "identificacion": "111",
      "mensaje": "Turno asignado: T1",
      "turno": "T1"
    },
    {
      "id": 2,
      "identificacion": "222",
      "mensaje": "Turno asignado: T2",
      "turno": "T2"
    }
  ]
}
```



```
{
  "status": "Notifications activo"
}
```

2.5. Reports Service

El Reports Service fue implementado como un microservicio encargado de consolidar información proveniente de los demás servicios del sistema y aplicar mecanismos de tolerancia a fallos mediante el patrón Circuit Breaker.

Este componente consulta información distribuida desde Users Service, Turns Service y Notifications Service utilizando comunicación HTTP/REST síncrona.

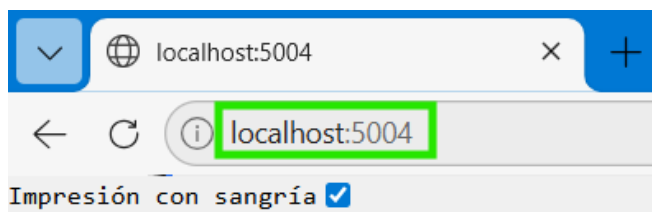
Adicionalmente, el servicio implementa persistencia desacoplada mediante PostgreSQL para almacenar historial de reportes generados, estados del circuito y datos de monitoreo básico.

Puerto utilizado: 5004

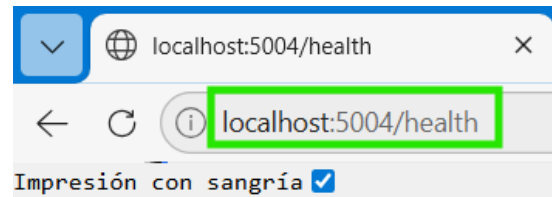
Base de datos asociada: reports_db

Endpoints implementados:

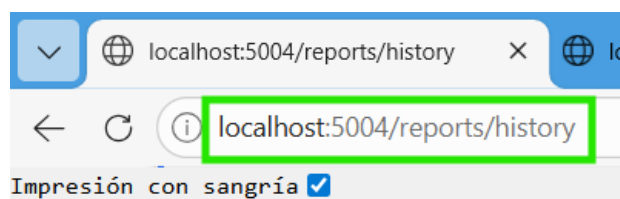
- GET /reports
- POST /reports
- GET /reports/history
- GET /health



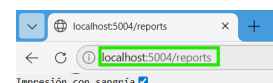
```
{
  "mensaje": "Reports Service activo"
}
```



```
{
  "status": "Reports activo"
}
```



```
{
  "historial": [
    {
      "estado_circuito": "CLOSED",
      "fecha": "2026-05-25 04:02:04.079799",
      "id": 1,
      "notificaciones": 2,
      "turnos": 2,
      "usuarios": 2
    }
  ]
}
```



```
{
  "estado_circuito": "CLOSED",
  "notificaciones": [
    {
      "id": 1,
      "identificacion": "111",
      "mensaje": "Turno asignado: T1",
      "turno": "T1"
    },
    {
      "id": 2,
      "identificacion": "222",
      "mensaje": "Turno asignado: T2",
      "turno": "T2"
    }
  ],
  "turnos": [
    {
      "id": 1,
      "identificacion": "111",
      "turno": "T1"
    },
    {
      "id": 2,
      "identificacion": "222",
      "turno": "T2"
    }
  ],
  "usuarios": [
    {
      "id": 1,
      "identificacion": "111",
      "telefono": "3215934303"
    },
    {
      "id": 2,
      "identificacion": "222",
      "telefono": "3215934300"
    }
  ]
}
```

Comunicación entre servicios

La comunicación implementada es de tipo síncrona utilizando HTTP/REST mediante la librería Requests de Python.

Las principales relaciones entre componentes son:

- Gateway → Todos los servicios
- Turns Service → Users Service
- Turns Service → Notifications Service
- Reports Service → Todos los servicios

El flujo principal de peticiones funciona de la siguiente manera:

1. El cliente realiza una solicitud al Gateway.
2. El Gateway redirecciona la petición al servicio correspondiente.
3. El microservicio procesa la solicitud.
4. Si es necesario, se comunica con otros servicios.
5. La respuesta retorna al Gateway y posteriormente al cliente

Persistencia y bases de datos

El sistema implementa persistencia desacoplada mediante PostgreSQL.

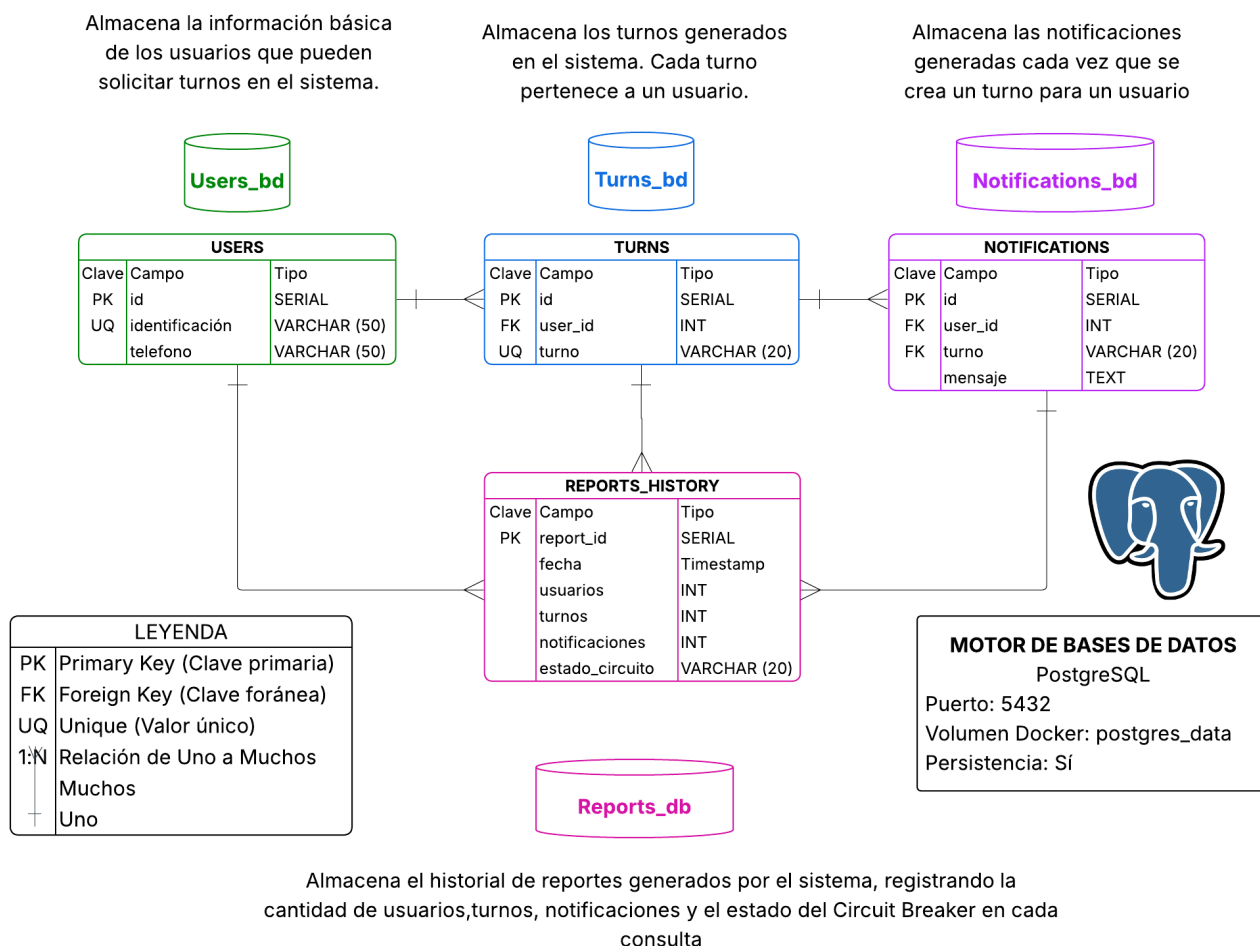
Cada microservicio administra su propia base de datos lógica independiente:

- users_db
- turns_db
- notifications_db
- reports_db

Esta estrategia permite reducir el acoplamiento entre componentes y facilitar futuras escalabilidades dentro de la arquitectura distribuida.

DIAGRAMA RELACIONAL DE BASES DE DATOS

Sistema de Gestión de Turnos - Arquitectura de Microservicios



3. JUSTIFICACIÓN DE LA ARQUITECTURA

La decisión de dividir el sistema en microservicios surgió como respuesta a las limitaciones presentes en arquitecturas monolíticas tradicionales. En un sistema monolítico, todas las funcionalidades se encuentran centralizadas dentro de una única aplicación, generando altos niveles de acoplamiento y dificultando el mantenimiento, la escalabilidad y la recuperación ante fallos.

Con el objetivo de mejorar la modularidad del sistema, se decidió separar cada funcionalidad principal en servicios independientes. Esta estrategia permitió distribuir las responsabilidades del sistema y reducir dependencias directas entre componentes.

El Users Service fue diseñado exclusivamente para administrar la información relacionada con usuarios. Su responsabilidad consiste en registrar y consultar usuarios dentro del sistema, manteniendo persistencia independiente y validaciones propias.

El Turns Service se implementó para concentrar únicamente la lógica relacionada con la generación automática de turnos. Este servicio coordina la validación de usuarios y el envío de notificaciones, convirtiéndose en uno de los principales puntos de integración de la arquitectura.

El Notifications Service fue desacoplado para separar completamente la lógica de notificaciones del resto de operaciones del sistema. Esto permitió mantener independencia funcional y reducir el impacto de posibles modificaciones futuras sobre otros componentes.

Por otro lado, el Reports Service se incorporó como un microservicio especializado en consolidar información distribuida y aplicar mecanismos de tolerancia a fallos. Su implementación permitió demostrar integración distribuida y aplicar el patrón Circuit Breaker dentro de un entorno real de microservicios.

Finalmente, el Gateway Service fue implementado para centralizar el acceso al sistema y desacoplar completamente al cliente de la arquitectura interna.

Entre las principales ventajas obtenidas mediante esta arquitectura se encuentra el desacoplamiento funcional de componentes, permitiendo que cada servicio evolucione de manera independiente. Adicionalmente, la arquitectura facilita el mantenimiento, mejora la organización lógica del sistema y reduce el impacto de posibles fallos internos.

Otra ventaja importante corresponde a la tolerancia parcial a fallos. Debido a que los servicios se encuentran desacoplados, un error interno no necesariamente afecta la totalidad del sistema. Esto pudo evidenciarse durante las pruebas realizadas mediante detención y recuperación de contenedores Docker.

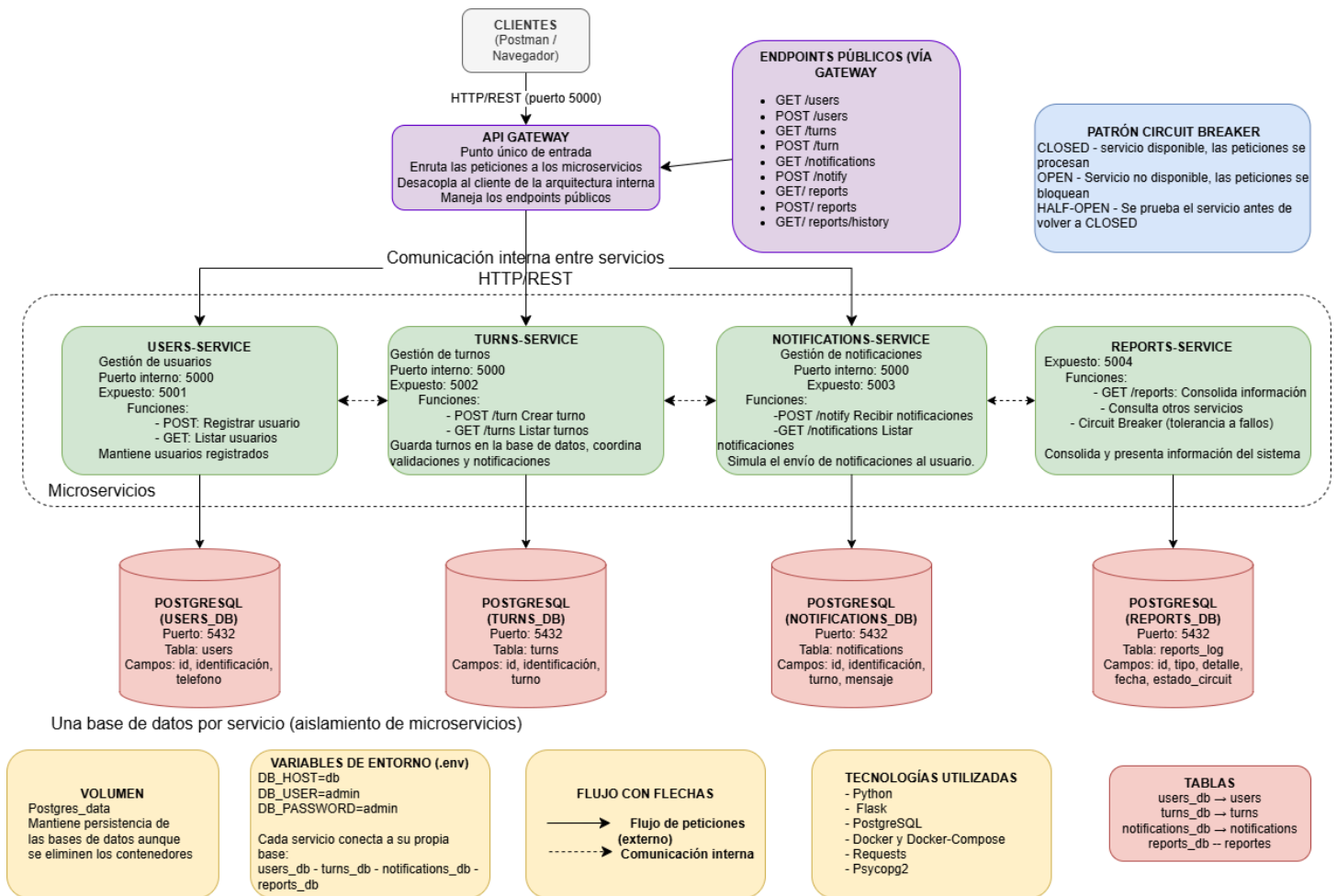
Sin embargo, durante el desarrollo también surgieron dificultades técnicas importantes. Una de las principales dificultades correspondió a la comunicación entre contenedores Docker y la sincronización de servicios distribuidos. También se presentaron retos

relacionados con la administración de dependencias, recuperación de servicios y validación correcta de estados del patrón Circuit Breaker.

Estas dificultades permitieron comprender de manera práctica los desafíos reales presentes en arquitecturas distribuidas modernas.

SISTEMA DE GESTIÓN DE TURNOS - ARQUITECTURA DE MICROSERVICIOS

Sistema distribuido basado en microservicios que permite registrar usuarios, generar turnos automáticamente, enviar notificaciones y consolidar información mediante el servicio de reportes



4. TOLERANCIA A FALLOS

Debido a que los sistemas distribuidos dependen de múltiples componentes ejecutándose simultáneamente, fue necesario implementar mecanismos de tolerancia a fallos que permitieran controlar errores y reducir el impacto de servicios indisponibles.

4.1. Manejo de errores

Cada microservicio implementa bloques try/except para capturar errores relacionados con:

- fallos de comunicación HTTP
- indisponibilidad de servicios
- timeouts
- errores internos

Esto permite responder mensajes controlados sin afectar completamente el sistema.

```
while True:
    try:
        conn = psycopg2.connect(
            host=os.getenv("DB_HOST"),
            database=os.getenv("DB_NAME"),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD")
        )

        cur = conn.cursor()

        logging.info( # type: ignore
            "Conectado a notifications_db"
        )
        break

    except:
        logging.error( # type: ignore
            "Esperando base de datos..."
        )

        time.sleep(3)
```

4.2. Circuit Breaker

El patrón Circuit Breaker fue implementado dentro del Reports Service y tiene como objetivo evitar solicitudes repetitivas hacia servicios caídos y reducir el consumo innecesario de recursos.

El circuito implementa tres estados:

- CLOSED → funcionamiento normal

- OPEN → servicio no disponible
- HALF-OPEN → validación de recuperación

Cuando un servicio falla múltiples veces consecutivas, el circuito cambia automáticamente a estado OPEN. Después de un tiempo de espera, el sistema prueba nuevamente la disponibilidad utilizando el estado HALF-OPEN.

```

users-service-1 exited with code 137
reports-service-1 | 2026-05-24 19:30:05,555 INFO REPORTS: Consultando users-service
reports-service-1 | 2026-05-24 19:30:12,725 ERROR REPORTS: Fallo detectado: HTTPConnectionPool(host='users-service', port=5000): Max retries exceeded with url: /users (Caused by NameResolutionError("HTTPConnection(host='users-service', port=5000): Failed to resolve 'users-service' ([Errno -2] Name or service not known)"))
reports-service-1 | 2026-05-24 19:30:12,726 ERROR REPORTS: Numero de fallos: 1
reports-service-1 | 2026-05-24 19:30:12,729 INFO REPORTS: 172.18.0.1 - - [24/May/2026 19:30:12] "GET /reports HTTP/1.1" 500 -
reports-service-1 | 2026-05-24 19:30:17,402 INFO REPORTS: Consultando users-service
reports-service-1 | 2026-05-24 19:30:24,528 ERROR REPORTS: Fallo detectado: HTTPConnectionPool(host='users-service', port=5000): Max retries exceeded with url: /users (Caused by NameResolutionError("HTTPConnection(host='users-service', port=5000): Failed to resolve 'users-service' ([Errno -2] Name or service not known)"))
reports-service-1 | 2026-05-24 19:30:24,528 ERROR REPORTS: Numero de fallos: 2
reports-service-1 | 2026-05-24 19:30:24,529 INFO REPORTS: 172.18.0.1 - - [24/May/2026 19:30:24] "GET /reports HTTP/1.1" 500 -
reports-service-1 | 2026-05-24 19:30:29,354 INFO REPORTS: Consultando users-service
reports-service-1 | 2026-05-24 19:30:36,516 ERROR REPORTS: Fallo detectado: HTTPConnectionPool(host='users-service', port=5000): Max retries exceeded with url: /users (Caused by NameResolutionError("HTTPConnection(host='users-service', port=5000): Failed to resolve 'users-service' ([Errno -2] Name or service not known)"))
reports-service-1 | 2026-05-24 19:30:36,516 ERROR REPORTS: Numero de fallos: 3
reports-service-1 | 2026-05-24 19:30:36,516 ERROR REPORTS: Circuito OPEN
reports-service-1 | 2026-05-24 19:30:36,517 INFO REPORTS: 172.18.0.1 - - [24/May/2026 19:30:36] "GET /reports HTTP/1.1" 500 -

```

4.3. Recuperación de servicios

La recuperación fue validada utilizando: **docker stop** y **docker start**. Esto permitió simular fallos reales dentro de la arquitectura distribuida.

```

PS C:\Users\ASUS\Desktop\gestor-turnos> & "C:\Program Files\Docker\Docker\resources\bin\docker.exe" stop gestor-turnos-users-service-1
gestor-turnos-users-service-1
PS C:\Users\ASUS\Desktop\gestor-turnos>

```

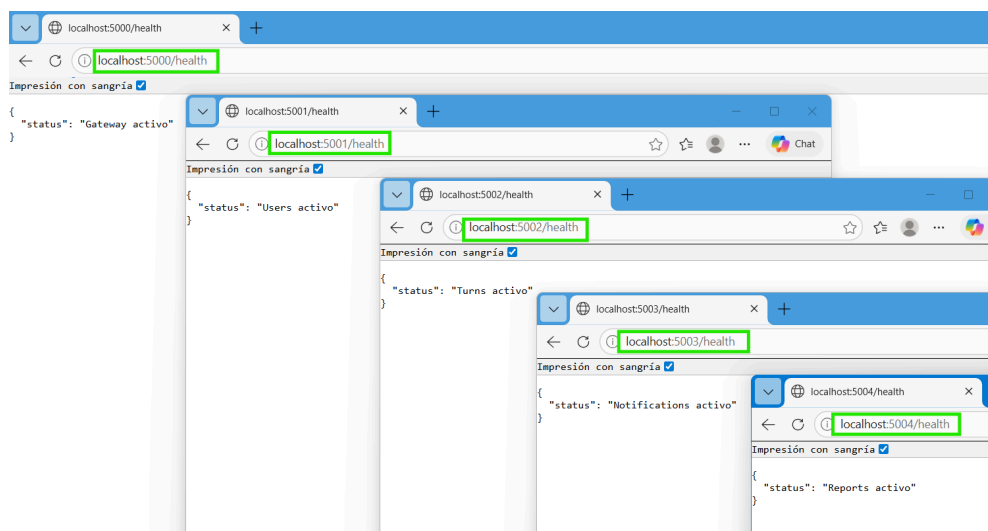
```

PS C:\Users\ASUS\Desktop\gestor-turnos> & "C:\Program Files\Docker\Docker\resources\bin\docker.exe" start gestor-turnos-users-service-1
gestor-turnos-users-service-1
PS C:\Users\ASUS\Desktop\gestor-turnos> |

```

4.4. Validación de disponibilidad

Cada microservicio implementa endpoints: **/health**. Estos endpoints permiten validar la disponibilidad y funcionamiento del sistema.



4.5. Logs y monitoreo básico

El sistema implementa logs distribuidos utilizando logging. Los logs permiten monitorear:

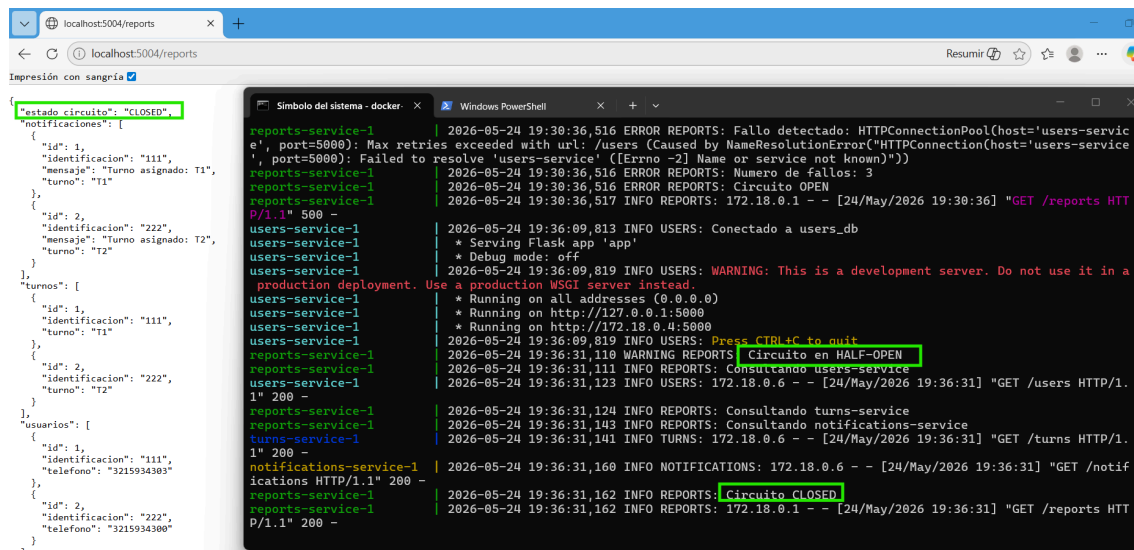
- solicitudes
- errores
- estados del circuito
- recuperación
- disponibilidad

```
notifications-service-1 | 2026-05-24 18:21:47,977 INFO NOTIFICATIONS: 172.18.0.1 - - [24/May/2026 18:21:47] "GET /favicon.ico HTTP/1.1" 404 -
notifications-service-1 | 2026-05-24 18:57:16,968 INFO NOTIFICATIONS: 172.18.0.5 - - [24/May/2026 18:57:16] "POST /notify HTTP/1.1" 200 -
notifications-service-1 | 2026-05-24 18:58:53,384 INFO NOTIFICATIONS: 172.18.0.7 - - [24/May/2026 18:58:53] "GET /notifications HTTP/1.1" 200 -
notifications-service-1 | 2026-05-24 19:08:24,729 INFO NOTIFICATIONS: 172.18.0.6 - - [24/May/2026 19:08:24] "GET /notifications HTTP/1.1" 200 -
turns-service-1 | * Serving Flask app 'app'
turns-service-1 | * Debug mode: off
turns-service-1 | 2026-05-24 17:18:39,370 INFO TURNS: WARNING: This is a development server. Do not use it in a production deployment. Use a produc
tion WSGI server instead.
turns-service-1 | * Running on all addresses (0.0.0.0)
turns-service-1 | * Running on http://127.0.0.1:5000
turns-service-1 | * Running on http://172.18.0.4:5000
turns-service-1 | 2026-05-24 17:18:39,370 INFO TURNS: Press CTRL+C to quit
turns-service-1 | 2026-05-24 18:21:39,555 INFO TURNS: 172.18.0.1 - - [24/May/2026 18:21:39] "GET /health HTTP/1.1" 200 -
turns-service-1 | 2026-05-24 18:21:39,614 INFO TURNS: 172.18.0.1 - - [24/May/2026 18:21:39] "GET /favicon.ico HTTP/1.1" 404 -
turns-service-1 | 2026-05-24 18:57:16,949 INFO TURNS: Turno creado T2 para 222
turns-service-1 | 2026-05-24 18:57:16,962 INFO TURNS: Notificacion enviada
turns-service-1 | 2026-05-24 18:57:16,962 INFO TURNS: 172.18.0.7 - - [24/May/2026 18:57:16] "POST /turn HTTP/1.1" 200 -
turns-service-1 | 2026-05-24 19:08:24,722 INFO TURNS: 172.18.0.6 - - [24/May/2026 19:08:24] "GET /turns HTTP/1.1" 200 -
gateway-service-1 | 172.18.0.1 - - [24/May/2026 18:21:39] "GET /favicon.ico HTTP/1.1" 404 -
gateway-service-1 | 172.18.0.1 - - [24/May/2026 18:28:52] "GET /health HTTP/1.1" 200 -
gateway-service-1 | 172.18.0.1 - - [24/May/2026 18:54:49] "GET /users HTTP/1.1" 200 -
gateway-service-1 | 172.18.0.1 - - [24/May/2026 18:55:15] "POST /users HTTP/1.1" 200 -
gateway-service-1 | 172.18.0.1 - - [24/May/2026 18:57:16] "POST /turn HTTP/1.1" 200 -
gateway-service-1 | 172.18.0.1 - - [24/May/2026 18:58:53] "GET /notifications HTTP/1.1" 200 -
users-service-1 | * Debug mode: off
users-service-1 | 2026-05-24 17:18:39,042 INFO USERS: WARNING: This is a development server. Do not use it in a production deployment. Use a produc
tion WSGI server instead.
users-service-1 | * Running on all addresses (0.0.0.0)
users-service-1 | * Running on http://127.0.0.1:5000
users-service-1 | * Running on http://172.18.0.4:5000
users-service-1 | 2026-05-24 17:18:39,042 INFO USERS: Press CTRL+C to quit
users-service-1 | 2026-05-24 18:21:24,668 INFO USERS: 172.18.0.1 - - [24/May/2026 18:21:24] "GET /health HTTP/1.1" 200 -
users-service-1 | 2026-05-24 18:21:24,714 INFO USERS: 172.18.0.1 - - [24/May/2026 18:21:24] "GET /favicon.ico HTTP/1.1" 404 -
users-service-1 | 2026-05-24 18:54:49,567 INFO USERS: 172.18.0.7 - - [24/May/2026 18:54:49] "GET /users HTTP/1.1" 200 -
users-service-1 | 2026-05-24 18:55:15,866 INFO USERS: Usuario creado 222
users-service-1 | 2026-05-24 18:55:15,866 INFO USERS: 172.18.0.7 - - [24/May/2026 18:55:15] "POST /users HTTP/1.1" 200 -
users-service-1 | 2026-05-24 18:57:16,935 INFO USERS: 172.18.0.5 - - [24/May/2026 18:57:16] "GET /users HTTP/1.1" 200 -
users-service-1 | 2026-05-24 19:08:24,714 INFO USERS: 172.18.0.6 - - [24/May/2026 19:08:24] "GET /users HTTP/1.1" 200 -
PS C:\Users\VASUS\Desktop\gestor-turnos> |
```

4.6. Ejemplos reales implementados

Durante las pruebas se realizaron las siguientes validaciones:

- detención del Users Service
- apertura del circuito
- recuperación automática
- cambio a HALF-OPEN
- retorno a CLOSED



```
{
  "estado_circuito": "CLOSED",
  "notificaciones": [
    {
      "id": 1,
      "identificacion": "111",
      "mensaje": "Turno asignado: T1",
      "turno": "T1"
    },
    {
      "id": 2,
      "identificacion": "222",
      "mensaje": "Turno asignado: T2",
      "turno": "T2"
    }
  ],
  "turnos": [
    {
      "id": 1,
      "identificacion": "111",
      "turno": "T1"
    },
    {
      "id": 2,
      "identificacion": "222",
      "turno": "T2"
    }
  ],
  "usuarios": [
    {
      "id": 1,
      "identificacion": "111",
      "telefono": "3215934303"
    },
    {
      "id": 2,
      "identificacion": "222",
      "telefono": "3215934300"
    }
  ]
}
```

```
reports-service-1 | 2026-05-24 19:30:36,516 ERROR REPORTS: Fallo detectado: HTTPConnectionPool(host='users-servic
e', port=5000): Max retries exceeded with url: /users (Caused by NameResolutionError("HTTPConnection(host='users-servic
e', port=5000): Failed to resolve 'users-service' ([Errno -2] Name or service not known)))
reports-service-1 | 2026-05-24 19:30:36,516 ERROR REPORTS: Numero de fallos: 3
reports-service-1 | 2026-05-24 19:30:36,516 ERROR REPORTS: Circuito OPEN
reports-service-1 | 2026-05-24 19:30:36,517 INFO REPORTS: 172.18.0.1 - - [24/May/2026 19:30:36] "GET /reports HTT
P/1.1" 500 -
users-service-1 | 2026-05-24 19:36:09,813 INFO USERS: Conectado a users_db
users-service-1 | * Serving Flask app 'app'
users-service-1 | * Debug mode: off
users-service-1 | 2026-05-24 19:36:09,819 INFO USERS: WARNING: This is a development server. Do not use it in a
production deployment. Use a production WSGI server instead.
users-service-1 | * Running on all addresses (0.0.0.0)
users-service-1 | * Running on http://127.0.0.1:5000
users-service-1 | * Running on http://172.18.0.4:5000
reports-service-1 | 2026-05-24 19:36:09,819 INFO REPORTS: Press CTRL+C to quit
reports-service-1 | 2026-05-24 19:36:31,110 WARNING REPORTS: Circuito en HALF-OPEN
reports-service-1 | 2026-05-24 19:36:31,111 INFO REPORTS: Consultando users-service
reports-service-1 | 2026-05-24 19:36:31,123 INFO USERS: 172.18.0.6 - - [24/May/2026 19:36:31] "GET /users HTTP/1.
1" 200 -
reports-service-1 | 2026-05-24 19:36:31,124 INFO REPORTS: Consultando turns-service
reports-service-1 | 2026-05-24 19:36:31,143 INFO REPORTS: Consultando notifications-service
reports-service-1 | 2026-05-24 19:36:31,141 INFO TURNS: 172.18.0.6 - - [24/May/2026 19:36:31] "GET /turns HTTP/1.
1" 200 -
notifications-service-1 | 2026-05-24 19:36:31,160 INFO NOTIFICATIONS: 172.18.0.6 - - [24/May/2026 19:36:31] "GET /notif
ications HTTP/1.1" 200 -
reports-service-1 | 2026-05-24 19:36:31,162 INFO REPORTS: Circuito CLOSED
reports-service-1 | 2026-05-24 19:36:31,162 INFO REPORTS: 172.18.0.1 - - [24/May/2026 19:36:31] "GET /reports HTT
P/1.1" 200 -
```

4.7. Endpoint de monitoreo

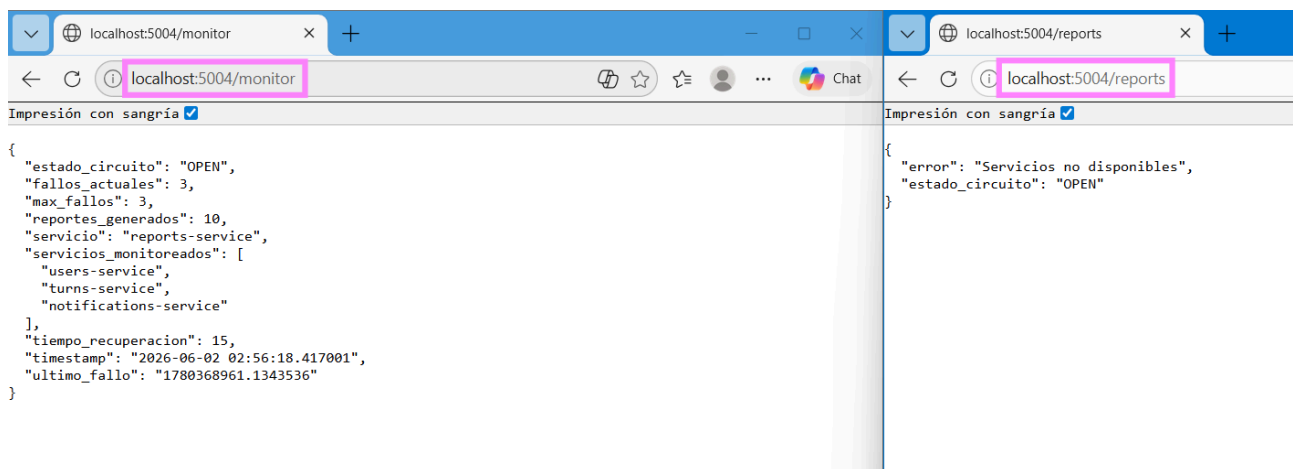
Como complemento a los mecanismos de tolerancia a fallos, se implementó un endpoint de monitoreo en el Reports Service que permite consultar en tiempo real el estado interno del sistema.

El endpoint `/monitor` expone métricas básicas relacionadas con el Circuit Breaker y la disponibilidad de los servicios monitoreados.

Entre los datos disponibles se encuentran:

- Estado actual del circuito (CLOSED, OPEN o HALF-OPEN).
- Cantidad de fallos registrados.
- Número máximo de fallos permitidos antes de abrir el circuito.
- Tiempo de recuperación configurado.
- Fecha y hora de la consulta.
- Cantidad de reportes generados.
- Servicios monitoreados por el sistema.

Esta información permite identificar rápidamente situaciones de error, validar el comportamiento del Circuit Breaker y verificar el estado operativo de la arquitectura distribuida.



5. ESCALABILIDAD Y ADMINISTRACIÓN

Actualmente el sistema no implementa balanceo de carga debido a que el proyecto se encuentra orientado a una implementación académica básica, sin embargo, la arquitectura fue diseñada considerando futuras mejoras relacionadas con escalamiento horizontal y administración distribuida.

Gracias al desacoplamiento funcional, cada microservicio podría escalarse individualmente dependiendo de la carga del sistema.

Entre las principales limitaciones actuales se encuentran:

- ausencia de balanceador de carga
- monitoreo básico
- autenticación limitada
- comunicación síncrona únicamente

Como posibles mejoras futuras se plantea implementar:

- balanceo de carga
- Kubernetes
- Prometheus y Grafana
- RabbitMQ
- autenticación JWT
- monitoreo avanzado

6. SEGURIDAD BÁSICA

El sistema implementa variables de entorno mediante archivos “.env.”

Esto permitió desacoplar información sensible del código fuente y facilitar la configuración del sistema.

Variables utilizadas: DB_HOST=db; DB_USER=admin; DB_PASSWORD=admin

Además, se implementó un archivo “.env.example” para compartir la estructura de configuración sin exponer credenciales reales.